۱-۶ امنیت و حفاظت

اگر یک سیستم کامپیوتری کاربران متعددی داشته و اجرای همروند چندین پردازه را فراهم می‌کند، در این صورت بایستی دسترسی به داده‌ها هماهنگ شود. در همین راستا، راهکارهایی هستند که اطمینان می‌دهند تنها پردازه‌هایی که از جانب سیستم‌عامل مجوز مناسبی کسب کرده باشند، می‌توانند روی فایل‌ها، قطعات حافظه، پردازنده و سایر منابع عملیات داشته باشند. برای مثال، سخت‌افزار آدرس‌دهی حافظه اطمنیان می‌دهد که یک پردازه فقط می‌تواند در فضای آدرس خودش اجرا داشته باشد. تایمر تضمین می‌نماید که هیچ پردازه‌ای نمی‌تواند کنترل پردازنده را تا ابد در اختیار بگیرد و بایستی آن را رها کند. ثبات‌های کنترل دستگاه در دسترس کاربران نیستند، لذا سلامت دستگاه‌های جانبی مختلف محفوظ است.

پس، **حفاظت** عبارت است از هرگونه راهکاری که برای کنترل دسترسی پردازه‌ها یا کاربران به منابع تعریف‌شده در یک سیستم کامپیوتری به کار می‌آید. این راهکار بایستی ابزاری برای توصیف کنترل‌های اعمالی و اجبار آنها فراهم کند.

حفاظت می‌تواند با تشخیص خطاهای پنهان در رابطهای میان زیرسیستم‌های جزء، قابلیت اطمینان را بهبود ببخشد. تشخیص زودهنگام خطاهای رابطها اغلب می‌تواند از آلودگی یک زیرسیستم سالم توسط زیرسیستم خرابکار دیگر جلوگیری کند. به علاوه، یک منبع فاقد حفاظت نمی‌تواند در مقابل استفاده‌ی (یا سوءاستفاده‌ی) یک کاربر غیرمجاز یا فاقد صلاحیت از خود دفاع کند. یک سیستم حفاظت‌دار، ابزاری برای تمییز استفاده‌ی مجاز و غیرمجاز فراهم می‌کند که این موضوع را در فصل ۱۷ با عمق بیشتری بحث خواهیم کرد.

یک سیستم می‌تواند به رغم داشتن حفاظت کافی، همچنان در معرض خرابی قرار داشته و اجازه‌ی دسترسی نامناسب را بدهد. کاربری را در نظر بگیرید که اطلاعات شناسایی هویتش (که به منزله‌ی ابزار شناسایی او برای سیستم است) ربوده می‌شود. داده‌های او ممکن است حتی با وجود داشتن حفاظت فایل و حافظه، کپی یا حذف شوند. این وظیفه‌ی **امنیت** است که از سیستم در مقابل تهاجمات داخلی و خارجی دفاع کند. چنین حملاتی از جناح‌های مختلف به صورت گسترده صورت می‌گیرند که ویروس‌ها و کرم‌ها، حملات ممانعت از سرویس[1] (که تمامی منابع سیستم را مشغول ساخته و مانع سرویس کاربران قانونی سیستم می‌شوند)، سرقت شناسه و سرقت سرویس (استفاده‌ی غیرمجاز از سیستم) از آن جمله‌اند. مقابله با برخی از این حملات در بعضی سیستم‌ها به عنوان یک کارکرد سیستم‌عامل محسوب می‌شود، حال آنکه سیستم‌های دیگری هستند که آن را به سیاست یا نرم‌افزار دیگری واگذار می‌کنند. به خاطر وقایع امنیتی تکان‌دهنده‌ای که وجود دارد، تحقیق و پیاده‌سازی امکانات امنیتی سیستم‌عامل‌ها حوزه‌ای است که رشد روزافزونی دارد. امنیت را در فصل ۱۶ بحث می‌کنیم.

حفاظت و امنیت ملزم می‌کند که سیستم بتواند کاربران خود را در میان همه تمییز بدهد. اکثر سیستم‌عامل‌ها فهرستی از نام کاربران و **شناسه‌های کاربری** مرتبط با آنها را حفظ می‌کنند. در

1. Denial of service

ادبیات ویندوز، این موضوع همان **شناسه‌ی امنیتی**[1] **(SID)** است. این شناسه‌های عددی برای هر کاربر یکتا هستند. زمانی که کاربری به سیستم وارد می‌شود، مرحله‌ی شناسایی هویت، شناسه مناسب کاربر را تعیین می‌کند. این شناسه‌ی کاربر به تمامی پردازه‌ها و نخ‌های کاربر منتسب می‌شود. زمانی که لازم می‌شود یک شناسه برای کاربری خوانا باشد، دوباره از طریق لیست نام کاربری به نام کاربر ترجمه می‌شود.

در بعضی شرایط نیاز است که به جای تک‌کاربر، مجموعه‌ای از کاربران را از بقیه متمایز کنیم. برای مثال، مالک یک فایل در سیستم یونیکس می‌تواند تمامی عملیات روی فایل را صادر کند، در حالی که ممکن است مجموعه‌ی منتخبی از کاربران، فقط مجاز به خواندن فایل باشند. برای انجام این کار لازم داریم که یک نام گروه و مجموعه‌ای از کاربران متعلق به آن گروه را تعریف کنیم. کارکرد گروه می‌تواند به صورت لیستی از اسامی گروه در سطح سیستم و **شناسه‌ی گروه‌ها** تعریف شود. یک کاربر، بسته به تصمیمات گرفته شده در طراحی سیستم‌عامل می‌تواند در یک یا چندین گروه قرار بگیرد. شناسه‌های گروه کاربر نیز به هر یک از پردازه‌ها و نخ‌های کاربر منتسب می‌شوند.

در روال معمول کار با یک سیستم، شناسه‌ی کاربر و شناسه‌ی گروه یک کاربر کافی است. با این وجود، گاهاً کاربر نیاز دارد تا امتیازات خود را بالا ببرد تا مجوزهای اضافه‌تری را برای یک فعالیت بدست آورد. برای مثال، کاربر ممکن است نیازمند دسترسی یک دستگاه محدودشده باشد. سیستم‌عامل‌ها ارتقای امتیاز را به روش‌های مختلفی فراهم می‌کنند. برای نمونه، در یونیکس، صفت setuid روی یک برنامه موجب می‌شود که آن برنامه به جای شناسه‌ی کاربر فعلی، با شناسه‌ی کاربر مالک فایل اجرا شود. پردازه با این **شناسه‌ی مؤثر**[2] اجرا می‌شود تا این که امتیازهای اضافی را رها کند یا خاتمه بپذیرد.

۱-۷ مجازی‌سازی

مجازی‌سازی یک فناوری است که به ما اجازه می‌دهد تا سخت‌افزار یک کامپیوتر منفرد (CPU، حافظه، درایوهای دیسک، کارت‌های رابط شبکه، و الی‌آخر) را به‌شکل چندین محیط اجرایی متفاوت انتزاعی سازیم و این تصور را به‌وجود بیاوریم که هر محیط جداگانه بر روی کامپیوتر اختصاصی خاص خودش اجرا می‌شود. این محیط‌ها را می‌توان به‌صورت سیستم‌عامل‌های متفاوت مشخصی دید (برای مثال، ویندوز و لینوکس) که می‌توانند همزمان اجرا شده و با یکدیگر تعامل داشته باشند. کاربر یک **ماشین مجازی** می‌تواند به‌همان شیوه‌ای که یک کاربر می‌تواند در میان پردازه‌های مختلف که در یک سیستم‌عامل منفردی به‌صورت همروند اجرا می‌شوند جابجا شود، در میان سیستم‌عامل‌های مختلف جابجا شود.

مجازی‌سازی به سیستم‌عامل‌ها اجازه می‌دهد تا به‌صورت کاربردهایی در میان سیستم‌عامل‌های دیگر اجرا شوند. در نگاه اول، به نظر می‌رسد که دلیل کافی برای چنین

1. Security ID
2. Effective UID

کارکردی وجود ندارد. اما صنعت مجازی‌سازی همه جا حضور داشته و در حال رشد است که شاهدی بر سودمندی و اهمیت آن است.

به طور کلی، مجازی‌سازی عضوی از یک رده نرم‌افزار است که مشمول تقلید[1] نیز می‌باشد. **تقلید** زمانی استفاده دارد که نوع پردازنده‌ی مبدأ با نوع پردازنده‌ی مقصد فرق دارد. برای مثال، موقعی که اپل[2] در کامپیوترهای رومیزی و لپ‌تاپ خود از پردازنده‌ی IBM Power به پردازنده‌ی اینتل x86 سوییچ می‌نماید، یک امکان تقلید به نام «Rosetta» لحاظ کرده است که امکان اجرای برنامه‌های کاربردی کامپایل شده برای پردازنده‌ی IBM را بر روی پردازنده‌ی اینتل فراهم می‌کند. همین مفهوم می‌تواند بسط داده شود تا امکان اجرای یک سیستم‌عامل کامل نوشته شده برای یک بستر را بر روی دیگری فراهم کند. با این وجود، تقلید، بهای سنگینی با خود به همراه دارد. هر دستورالعمل سطح ماشین که به صورت بومی بر روی سیستم مبدأ اجرا می‌شود بایستی به کارکرد معادل آن بر روی سیستم مقصد ترجمه شود که در بیشتر موارد منجر به چندین دستورالعمل مقصد می‌شود. اگر پردازنده‌های مبدأ و مقصد دارای سطوح کارایی مشابهی باشند، کد تقلیدی می‌تواند بسیار کندتر از کد بومی اجرا شود.

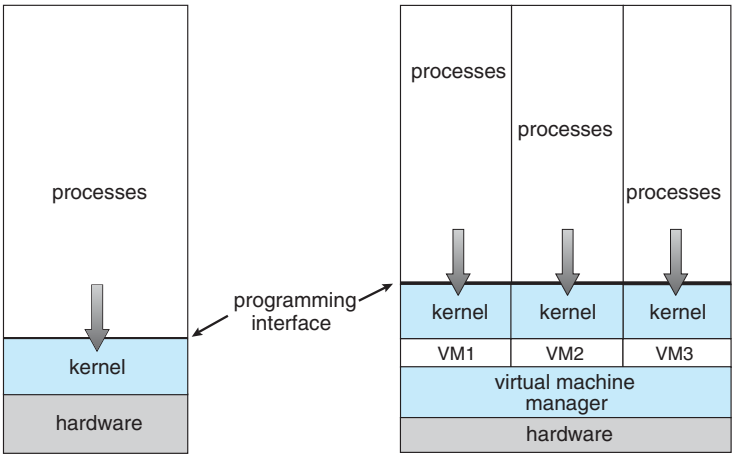
در مقایسه، سیستم‌عاملی که به صورت بومی روی یک معماری پردازنده‌ی خاص کامپایل می‌شود با **مجازی‌سازی** در میان سیستم‌عامل دیگر که آن هم بومی آن پردازنده است اجرا می‌شود. مجازی‌سازی نخست بر روی کامپیوترهای بزرگ IBM، به عنوان روشی برای اجرای همروند وظایف کاربران متعدد ظاهر شد. اجرای چندین ماشین مجازی به کاربران بسیاری این اجازه را می‌داد (و هنوز هم امکان می‌دهد) که وظایفی را روی سیستمی به اجرا بگذارند که برای یک کاربر منفرد طراحی شده بود. بعدها، در جواب مسائلی که اجرای برنامه‌های کاربردی متعدد مایکروسافت ویندوز XP بر روی پردازنده‌ی اینتل x86 داشت، VMware یک فناوری مجازی‌سازی جدید، در قالب یک برنامه‌ی کاربردی که بر روی ویندوز اجرا می‌شد، خلق کرد. این برنامه‌ی کاربردی، یک یا چند کپی **میهمان** ویندوز یا سایر سیستم‌عامل‌های بومی x86 را اجرا می‌کرد که هر یک، برنامه‌های کاربردی خود را اجرا می‌کردند. (شکل ۱-۱۶ را ببینید.) ویندوز، سیستم‌عامل **میزبان** بود و برنامه‌ی کاربردی VMware، **مدیر ماشین مجازی**[3] (**VMM**) بود. VMM، سیستم‌عامل‌های میهمان را به اجرا گذاشته، استفاده‌ی منبع آنها را مدیریت کرده و هر یک از میهمان‌ها را از سایرین محافظت می‌کرد.

هرچند که سیستم‌عامل‌های مدرن قابلیت اجرای برنامه‌های کاربردی متعدد را به صورت کامل دارند، استفاده از مجازی‌سازی به رشد خود ادامه می‌دهد. VMM در لپ‌تاپ‌ها و کامپیوترهای رومیزی به کاربر اجازه می‌دهد تا چندین سیستم‌عامل را برای بررسی نصب کند یا کاربردهایی اجرا کند که برای سیستم‌عامل‌هایی غیر از میزبان بومی نوشته شده‌اند. برای مثال، لپ‌تاپ اپل که macOS را بر روی پردازنده‌ی x86 اجرا می‌کند، می‌تواند یک میهمان ویندوز ۱۰ اجرا نماید که امکان اجرای برنامه‌های ویندوز را فراهم کند. شرکت‌هایی که برای

1. Emulation
2. Apple
3. Virtual machine manager (VMM)

سیستم‌عامل‌های متعدد نرم‌افزار می‌نویسند، می‌توانند برای توسعه، آزمون و خطایابی از مجازی‌سازی استفاده کنند تا تمامی سیستم‌عامل‌ها را بر روی یک کارگزار فیزیکی منفرد اجرا کنند. مجازی‌سازی در مراکز داده‌ای، یک روش مرسوم اجرا و مدیریت محیط‌های محاسباتی شده است. VMMهایی چون VMware ESX و Citrix XenServer، دیگر بر روی سیستم‌عامل‌های میزبان اجرا نمی‌شوند، بلکه خود میزبان هستند که سرویس‌ها و مدیریت منبع را برای پردازه‌های ماشین مجازی فراهم می‌کنند.

ما با این کتاب، ماشین مجازی لینوکس را فراهم می‌کنیم که به شما امکان اجرای لینوکس را—در کنار ابزار توسعه‌ای که فراهم می‌کنیم—فارغ از این که سیستم‌عامل میزبان شما چیست بر روی سیستم شخصی شما می‌دهد. جزئیات بیشتر امکانات و پیاده‌سازی مجازی‌سازی را می‌توانید در فصل ۱۸ بیابید.



شکل ۱-۱۶ یک کامپیوتری که (a) یک تک‌سیستم‌عامل را اجرا می‌کند (b) سه ماشین مجازی را اجرا می‌کند.

۱-۸ سیستم‌های توزیع‌شده

سیستم توزیع‌شده، مجموعه‌ای از سیستم‌های کامپیوتری فیزیکی مجزا و احتمالاً ناهمگن است که برای تدارک دسترسی کاربران به منابع مختلفی که سیستم نگه می‌دارد، شبکه می‌شوند. این سیستم، دسترسی به یک منبع مشترک، سرعت محاسبات، کارکرد، دسترس‌پذیری داده‌ها و قابلیت اطمینان را افزایش می‌دهد. برخی سیستم‌عامل‌ها، دسترسی به شبکه را به شکلی از دسترسی فایل، با لحاظ جزئیات شبکه در درایور دستگاه رابط شبکه عمومیت می‌بخشند. سیستم‌های دیگری هم هستند که کاربران را مجبور می‌کنند تا کارکردهای شبکه را مشخصاً احضار کنند. در حالت کلی، سیستم‌ها تلفیقی از این دو مد را دارا هستند—برای مثال، FTP و NFS. پروتکل‌هایی که سیستم توزیع‌شده را به وجود می‌آورند می‌توانند تأثیر شگرفی در

سودمندی و فراگیری آن داشته باشند.

شبکه، با ساده‌ترین بیان، یک مسیر ارتباطی میان دو یا چند سیستم است. کارکرد سیستم‌های توزیع‌شده به شبکه‌بندی آنها بستگی دارد. شبکه‌ها بر اساس پروتکل‌های استفاده شده، فواصل میان گره‌ها و رسانه‌های انتقال متفاوتند. TCP/IP رایج‌ترین پروتکل شبکه است که معماری پایه‌ای اینترنت را فراهم می‌کند. اکثر سیستم‌عامل‌ها اعم از تمامی سیستم‌عامل‌های همه‌منظوره، TCP/IP را پشتیبانی می‌کنند. برخی سیستم‌ها به تناسب نیازشان، پروتکل‌های مناسب را پشتیبانی می‌کنند. از منظر سیستم‌عامل، پروتکل شبکه، خیلی ساده به یک دستگاه رابط –برای مثال، کارت شبکه –با یک درایور دستگاه برای مدیریت آن، همین‌طور نرم‌افزاری برای رسیدگی به داده‌ها نیاز دارد. این مفاهیم در سطح کتاب بحث می‌شود.

شبکه‌ها بر اساس فواصل میان گره‌های آنها مشخص می‌شوند. **شبکه‌ی محلی**[1] **(LAN)** کامپیوترها را در یک اتاق، ساختمان یا دانشکده به همدیگر متصل می‌کند. **شبکه‌ی گسترده**[2] **(WAN)** معمولاً ساختمان‌ها، شهرها یا کشورها را به همدیگر متصل می‌کند. یک شرکت بزرگ جهانی ممکن است برای اتصال و پوشش دفاتر خود در سطح دنیا یک شبکه‌ی گسترده به وجود بیاورد. این شبکه‌ها می‌توانند یک یا چندین پروتکل را اجرا کنند. تداوم ظهور فناوری‌های جدید، اشکال جدیدی از شبکه‌ها را به وجود می‌آورد. برای مثال، **شبکه‌ی شهری**[3] **(MAN)** می‌تواند ساختمان‌هایی را در یک شهر به هم متصل کند. دستگاه‌های بلوتوث و 802.11 می‌توانند ارتباط را در فواصل چندین فوت با استفاده از فناوری بی‌سیم برقرار کرده، در اصل **شبکه‌ای در حوزه‌ی شخصی**[4] **(PAN)** میان یک گوشی و یک هدست یا یک گوشی‌هوشمند و یک کامپیوتر رومیزی ایجاد کنند.

رسانه‌هایی که حامل شبکه‌ها هستند نیز متنوعند. آنها سیم‌های مسی، فیبر نوری و انتقالات بی‌سیم میان ماهواره‌ها، بشقاب‌های میکروویو و امواج رادیویی را شامل می‌شوند. زمانی که دستگاه‌های محاسباتی به تلفن‌های همراه متصل می‌شوند یک شبکه ایجاد می‌کنند. حتی با یک اشعه‌ی مادون قرمز کوتاه هم می‌توان یک شبکه‌ی ارتباطی ایجاد کرد. در سطح ابتدایی، هر بار که کامپیوترها با هم ارتباط دارند یک شبکه ایجاد کرده یا از آن استفاده می‌کنند. این شبکه‌ها نیز در کارایی و قابلیت اعتمادشان فرق دارند.

برای برخی سیستم‌عامل‌ها، مفهوم شبکه‌ها و سیستم‌های توزیع‌شده فراتر از تدارک یک اتصال شبکه است. **سیستم‌عامل شبکه** سیستم‌عاملی است که امکاناتی همچون اشتراک فایل را در شبکه به همراه یک طرح ارتباطی مهیا می‌کند تا امکان تبادل پیغام‌ها را میان پردازه‌های مختلف کامپیوترهای متفاوت فراهم کند. کامپیوتری که سیستم‌عامل شبکه را اجرا می‌کند، مستقل از سایر کامپیوترهای شبکه عمل می‌کند، اگرچه از شبکه آگاه بوده و قادر است با سایر کامپیوترهای شبکه نیز ارتباط برقرار کند. سیستم‌عامل توزیع‌شده، بستری با استقلال کمتر فراهم می‌کند.

1. Local-area network
2. Wide-area network
3. Metropolitan-area network
4. Personal-area network

کامپیوترهای متفاوت آن چنان ارتباط نزدیکی دارند که گویی تنها یک سیستم‌عامل است که شبکه را کنترل می‌کند. شبکه‌های کامپیوتری و سیستم‌های توزیع‌شده را در فصل ۱۹ پوشش می‌دهیم.

۱-۹ ساختمان‌داده‌های هسته

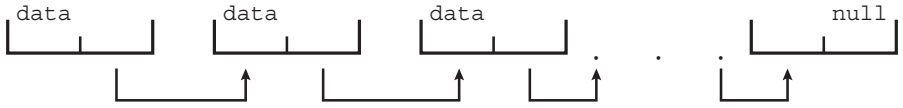
در ادامه به یک مبحث محوری در پیاده‌سازی سیستم‌عامل می‌پردازیم: شیوه‌ای که داده‌ها در سیستم ساختاردهی می‌شوند. در این بخش، چندین ساختمان‌داده‌ی مبنایی را به‌اختصار مطرح می‌کنیم که به صورت گسترده در سیستم‌عامل‌ها به کار گرفته می‌شوند.

۱-۹-۱ لیست، پشته و صف

آرایه ساختمان‌داده‌ی ساده‌ای است که هر عنصر آن می‌تواند به صورت مستقیم دستیابی شود. برای مثال، حافظه‌ی اصلی به صورت یک آرایه بنا می‌شود. اگر عنصری که ذخیره می‌شود بزرگ‌تر از یک بایت باشد، در این صورت چندین بایت می‌تواند به عنصر تخصیص یابد و عنصر به صورت شماره‌ی عنصر ضربدر اندازه‌ی عنصر آدرس‌دهی می‌شود. اما ذخیره و حذف عنصری که اندازه‌ی متغیری دارد چگونه خواهد بود؟ در صورتی که موقعیت‌های نسبی عناصر باقیمانده بایستی حفظ شوند؟ در چنین شرایطی، آرایه‌ها جای خود را به ساختمان‌داده‌های دیگری می‌دهند.

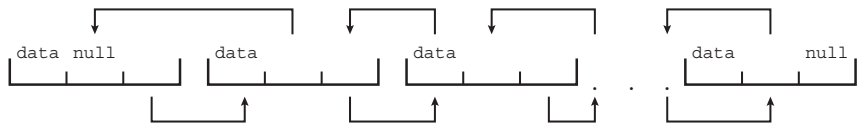
بعد از آرایه‌ها، شاید لیست‌ها رایج‌ترین ساختمان‌داده‌ها در علم کامپیوتر باشند. در حالی که هر عنصر آرایه می‌تواند به صورت مستقیم دستیابی شود، عناصر لیست بایستی با یک ترتیب خاص دسترسی شوند. یعنی، **لیست** معرف مجموعه‌ای از مقادیر داده‌ای به صورت یک دنباله است. رایج‌ترین روش پیاده‌سازی این ساختار، **لیست پیوندی** است که عناصر آن به یکدیگر پیوند داده می‌شوند. انواع متعدد لیست‌های پیوندی عبارتند از:

- ***لیست پیوندی یک طرفه***، که هر عنصر به بعدی آن اشاره می‌کند، چنان که در شکل ۱-۱۷ نشان داده می‌شود.



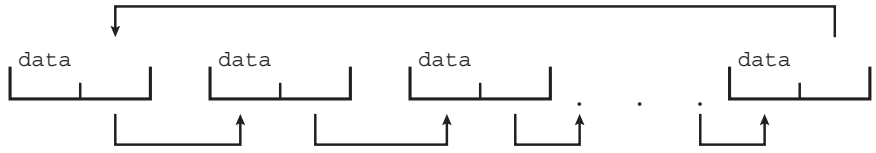
شکل ۱-۱۷ لیست پیوندی یک‌طرفه.

- ***لیست پیوندی دوطرفه***، که یک عنصر مشخص می‌تواند به قبلی و یا بعدی خود اشاره کند، چنان که در شکل ۱-۱۸ نشان داده می‌شود.



شکل ۱-۱۸ لیست پیوندی دوطرفه.

- ***لیست پیوندی چرخشی***، که عنصر آخر لیست به جای null، به عنصر اول اشاره می‌کند، چنان که در شکل ۱-۱۹ نشان داده می‌شود.



شکل ۱-۱۹ لیست پیوندی چرخشی.

لیست‌های پیوندی، عناصری را با اندازه‌های متغیر پوشش داده و امکان درج و حذف آسان عناصر را مقدور می‌سازند. یک عیب بالقوه‌ی استفاده از لیست در کارایی بازیابی یک عنصر مشخص است که در لیستی به اندازه‌ی n خطی است—$O(n)$، چرا که به‌صورت بالقوه لازم است در بدترین حالت، تمامی عناصر پیمایش شوند—خطی است. لیست‌ها گاهاً به صورت مستقیم توسط الگوریتم‌های هسته مورد استفاده قرار می‌گیرند. هرچند، به کرات برای ساخت ساختمان‌داده‌های پرقدرت، از قبیل پشته‌ها و صف‌ها به کار گرفته می‌شوند.

پشته یک ساختمان‌داده‌ی مرتب ترتیبی است که برای افزودن و حذف عناصر از اصل آخرین ورود، اولین خروج[1] (**LIFO**) استفاده می‌کند، با این مضمون که آخرین عنصری که به پشته وارد می‌شود اولین عنصری است که برداشته می‌شود. عملیات درج و حذف عناصر پشته، به ترتیب، به ***push*** و ***pop*** معروفند. سیستم‌عامل اغلب در موقع احضار فراخوانی‌های تابع از پشته استفاده می‌کند. پارامترها، متغیرهای محلی و آدرس برگشتی در زمان فراخوانی تابع در پشته گذاشته می‌شوند؛ در بازگشت از فراخوانی تابع، آن عناصر از پشته برداشته می‌شوند.

صف، در مقابل، یک ساختمان‌داده‌ی مرتب ترتیبی است که از اصل اولین-ورود، اولین-خروج[2] (**FIFO**) استفاده می‌کند: با این مضمون که عناصر صف، به ترتیبی که در صف درج شده‌اند، از صف حذف می‌شوند. نمونه‌های روزمره‌ی زیادی از صف‌ها وجود دارند، اعم از خریدارانی که در یک ردیف در یک فروشگاه به انتظار حساب سبد کالای خود هستند و خودروهایی که به ردیف در پشت چراغ راهنمایی منتظرند. صف‌ها در سیستم‌عامل‌ها نیز فراوان یافت می‌شوند–برای مثال، کارهایی که به یک چاپگر ارسال می‌شوند نوعاً به ترتیبی که ارسال شده‌اند، چاپ می‌شوند. همان‌گونه که در فصل ۵ خواهیم دید، وظایفی که به انتظار اجرا بر روی یک پردازنده‌ی در دسترس هستند، غالباً در صف‌هایی سازمان‌دهی می‌شوند.

۱-۹-۲ درخت

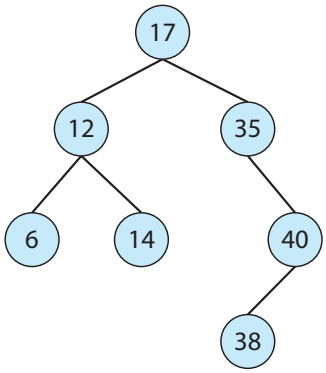
درخت ساختمان‌داده‌ای است که می‌تواند برای نمایش داده‌ها به صورت سلسله‌مراتبی استفاده شود. مقادیر داده‌ها در یک ساختار درخت از طریق روابط والد–فرزند متصل می‌شوند. در **درخت عمومی**، یک والد می‌تواند تعداد نامحدودی فرزند داشته باشد. در **درخت دودویی**، یک والد

1. Last-in first-out
2. First-in first-out

می‌تواند حداکثر دو فرزند داشته باشد که در اصطلاح، ***فرزند چپ*** و ***فرزند راست*** نامیده می‌شوند. به علاوه، **درخت جستجوی دودویی** نیازمند ترتیبی هم میان دو فرزند پدر می‌باشد، طوری که ***فرزند راست ≤ فرزند چپ***. شکل ۱-۲۰، نمونه‌ای از درخت جستجوی دودویی را نشان می‌دهد. موقعی که عنصری را در درخت جستجوی دودویی جستجو می‌کنیم، کارایی بدترین حالت آن، برابر O(n) است (فکر کنید که در چه حالتی، این پیچیدگی رخ می‌دهد). برای رفع این وضعیت می‌توانیم از یک الگوریتم استفاده کنیم تا **درخت جستجوی دودویی متوازنی** ایجاد کنیم. در اینجا، یک درخت شامل n عنصر، حداکثر دارای $lg\ n$ سطح است، لذا کارایی بدترین حالت آن، برابر O(lg n) تضمین می‌شود. در بخش ۵-۷-۱ خواهیم دید که لینوکس در بخشی از الگوریتم زمانبندی پردازنده‌اش از درخت جستجوی دودویی متوازن (معروف به **درخت قرمز-سیاه**) استفاده می‌کند.

۱-۹-۳ توابع درهم و نگاشت

تابع درهم[1] داده‌ها را به عنوان ورودی می‌گیرد، یک عملیات عددی بر روی این داده‌ها انجام داده و یک مقدار عددی برمی‌گرداند. سپس، این مقدار عددی می‌تواند برای بازیابی سریع داده‌ها به عنوان یک اندیس به یک جدول (نوعاً آرایه) استفاده شود. در حالی که جستجوی یک عنصر در لیستی به اندازه‌ی n می‌تواند در بدترین حالت به تعداد O(n) مقایسه نیاز داشته باشد، استفاده از تابع درهم برای بازیابی داده‌های جدول می‌تواند، بسته به جزئیات پیاده‌سازی، در بدترین حالت O(1) زمان ببرد. به خاطر این کارایی است که توابع درهم به صورت وسیعی در سیستم‌عامل‌ها به کار گرفته می‌شوند.



شکل ۱-۲۰ درخت جستجوی دودویی.

یکی از مشکلات بالقوه در رابطه با توابع درهم این است که دو ورودی یکتا می‌توانند مقدار خروجی یکسانی نتیجه بدهند–یعنی، می‌توانند به مکان یکسانی در جدول نگاشته شوند. می‌توانیم این وضعیت *برخورد درهم*[2] را با لحاظ یک لیست پیوندی در همان مکانی که تمامی این عناصر را

1. Hash function
2. Hash collision

با مقدار درهم یکسان در بر می‌گیرد، رفع کنیم. البته، هر اندازه برخوردها بیشتر باشند، کارآمدی تابع درهم نیز کمتر است.

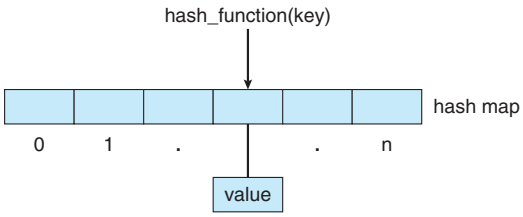
یک کاربرد تابع درهم در پیاده‌سازی **نقشه‌ی درهم** است که زوج‌های [کلید:مقدار] را با استفاده از یک تابع درهم به هم مرتبط می‌کند (یا *نگاشت می‌دهد*). همین که نگاشت برپا شد، می‌توانیم تابع درهم را به کلید اعمال کنیم تا مقداری را از نقشه‌ی درهم بدست آوریم (شکل ۱-۲۱). برای مثال، فرض کنید که نام کاربری به یک گذرواژه نگاشته می‌شود. سپس، احراز هویت گذرواژه به این صورت پیش می‌رود: کاربر، نام کاربری و گذرواژه‌ی خود را وارد می‌کند. تابع درهم به نام کاربری اعمال می‌شود که در ادامه برای بازیابی گذرواژه استفاده می‌شود. سپس، گذرواژه‌ی بازیابی شده، برای احراز هویت، با گذرواژه‌ی ورودی کاربر مقایسه می‌شود.

۱-۹-۴ نقشه‌های بیتی

نقشه‌ی بیتی، رشته‌ای از n رقم دودویی است که می‌تواند برای نمایش وضعیت n عنصر استفاده شود. برای مثال، فرض کنید منابع متعددی داریم و موجود بودن هر یک از منابع توسط مقدار یک رقم دودویی مشخص می‌شود: ۰ به معنی این که منبع موجود است، در حالی که ۱ نشان می‌دهد که در دسترس نیست (یا برعکس). مقدار iامین مکان در نقشه‌ی بیتی با منبع iام مرتبط است. به عنوان یک مثال، نقشه‌ی بیتی نشان داده شده در زیر را در نظر بگیرید:

0 0 1 0 1 1 1 0 1

منابع ۲، ۴، ۵، ۶ و ۸ در دسترس نیستند؛ منابع ۰، ۱، ۳ و ۷ موجودند.



شکل ۱-۲۱ نقشه‌ی درهم.

قدرت نقشه‌های بیتی وقتی نمایان می‌شود که به کارآمدی فضای آنها نگاهی بیندازیم. در صورتی که به جای فقط یک بیت از یک مقدار Boolean هشت بیتی استفاده می‌کردیم، ساختمان‌داده‌ی حاصل، هشت مرتبه بزرگ‌تر می‌شد. از این رو، نقشه‌های بیتی به صورت رایج در موقعی که لازم است در دسترس بودن تعداد زیادی منبع را نمایش بدهیم، استفاده می‌شوند. دیسک‌گردان‌ها یک تصویر عالی در این مورد هستند. یک دیسک‌گردان با اندازه‌ی متوسط ممکن است به چندین هزار واحد منفرد به نام **بلاک‌های دیسک** تقسیم شود. یک نقشه‌ی بیتی می‌تواند برای نشان دادن در دسترس بودن هر یک از بلاک‌های دیسک مورد استفاده قرار بگیرد.

به‌طور خلاصه، ساختمان‌داده‌ها در پیاده‌سازی‌های سیستم‌عامل فراوان یافت می‌شوند. بنابراین، ساختارهایی را که در اینجا بحث کردیم به همراه ساختارهای دیگر، همچنان که الگوریتم‌های هسته و پیاده‌سازی‌های آنها را بررسی می‌کنیم در کل این کتاب خواهیم دید.